One small bug in your app... and your entire system goes down.

Why does this happen?

Introduction to System Architectures

- Defines how an application is structured and organized
- Controls how components communicate and interact
- Impacts scalability, performance, maintainability

What is Monolithic Architecture?

- Single large codebase
- All modules are tightly coupled
- Uses single shared database
- Easy to start and develop
- Hard to scale and maintain
- One failure can affect entire system

What is Microservices Architecture?

- Application split into small independent services
- Each service handles one business function
- Uses database-per-service approach
- Services communicate via:
- Synchronous: REST / gRPC
- Asynchronous: Message brokers (Kafka, RabbitMQ)
- Independently scalable and deployable
- More complex to manage

Monolith vs Microservices

- **Monolith:** Single unit | **Microservices:** Multiple services
- **Monolith:** Shared DB | **Microservices:** Separate DBs
- **Monolith:** Tightly coupled | **Microservices:** Loosely coupled
- **Monolith:** Single deployment | **Microservices:** Independent deployments

Scaling & Deployment Comparison

- **Monolith:** Scale entire application
- **Microservices:** Scale only required service
- **Monolith:** One change → full redeployment
- **Microservices:** Change → deploy single service

Real-World Story: Netflix Migration

- Faced scaling and downtime issues
- Monolith couldn’t handle high traffic
- Migrated to microservices on cloud
- Improved scalability and reliability

Real-World Story: Amazon Scaling Challenges

- Monolith caused slow deployments
- Teams were tightly dependent
- Shifted to microservices
- Enabled independent teams and faster scaling

Real-World Story: Uber System Evolution

- Rapid growth caused performance bottlenecks
- Monolith became hard to manage
- Moved to microservices
- Achieved better performance and flexibility

When to Use Monolith vs Microservices

- Monolith: Small apps, simple requirements
- Microservices: Large apps, high scalability needs
- Monolith: Small teams
- Microservices: Multiple teams, complex systems

Common Mistakes Developers Make

- Using microservices for small/simple projects
- Ignoring DevOps, monitoring, and logging
- Poor service design (tight coupling)
- Not planning communication patterns properly
- Ignoring distributed system challenges:
- Network latency
- Data consistency issues
- Debugging complexity